
Departamento de Energía y Automatización

Robótica Industrial

PRÁCTICA Nº 1

Introducción al Arduino

Asignatura: Robótica Industrial (FPTEN27)

Profesores: José Barriola · Leonardo Delgado · José Montilva

Laboratorio de Robótica Industrial

Caracas, 23 de enero de 2026

1. Objetivo de la Práctica

El objetivo de esta práctica es familiarizarse con el sistema Arduino y su uso como unidad de supervisión y control. Para tal fin, se manipularán señales digitales de entrada (Di), salida (Do) y señales analógicas de entrada (Ai), modulación por ancho de pulso (PWM) y la comunicación serial con un terminal.

2. Configuración del Entorno de Desarrollo

2.1. Descarga del IDE de Arduino

Descargue la última versión del entorno integrado de desarrollo (IDE) desde la página oficial: [Arduino IDE — Download and Install](#).

2.2. Conexión de la Tarjeta Arduino

Las tarjetas Arduino Uno, Mega, Duemilanove y Nano se alimentan automáticamente a través del puerto USB de la computadora. En el caso de tarjetas más antiguas con puerto RS-232, se requerirá una fuente externa de alimentación entre 9 y 12 VDC.

2.3. Instalación del Driver USB

Instale el driver del Arduino correspondiente al sistema operativo en uso (Windows 7/10/11, macOS o Linux).

Nota sobre placas no oficiales

Si utiliza un Arduino **original**, el sistema operativo lo detectará automáticamente. Si la placa es **no original** (clones con chip CH340), descargue e instale manualmente el driver **CH340** desde el sitio del fabricante.

Procedimiento de verificación en Windows:

1. Abra el Menú Inicio y seleccione el Panel de Control.
2. Seleccione Sistema y, dentro de él, abra el Administrador de Dispositivos.
3. En la sección Puertos (COM y LPT) deberá aparecer un puerto con el nombre Arduino UNO (COMxx). Si no aparece allí, búsquelo en Otros Dispositivos como Dispositivo Desconocido.
4. Si requiere actualizar el driver, haga clic derecho sobre el puerto y seleccione Actualizar Software del Driver.

5. En el caso de Arduinos no originales, el dispositivo se identificará como COM CH340.
1. En equipos Apple, el puerto aparecerá con una nomenclatura distinta (por ejemplo, /dev/cu.usbmodem...).
2. Si el sistema solicita el archivo del driver, ubique el archivo arduino.inf dentro de la carpeta de drivers del IDE de Arduino. No utilice el driver FTDI USB.



Figura 1. Tarjeta Arduino UNO. Fuente: Arduino.cc

3. Primer Uso del IDE de Arduino

3.1. Ejecución de la Aplicación

Abra la aplicación Arduino IDE desde el menú de aplicaciones de su sistema operativo. El icono característico es el siguiente:



Figura 2. Icono del IDE de Arduino.

3.2. Selección de la Tarjeta

Vaya al menú **Herramientas** → **Tarjeta** y seleccione el modelo correspondiente a su hardware (por ejemplo, **Arduino Uno**).

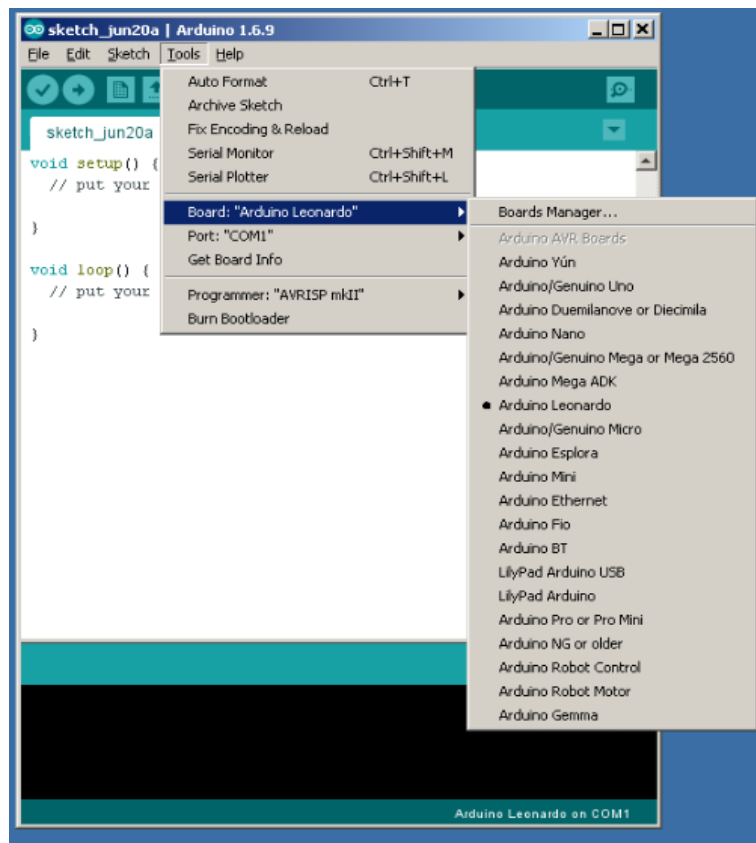


Figura 3. Selección de la tarjeta en el IDE.

3.3. Selección del Puerto Serial

Vaya al menú **Herramientas** → **Puerto Serial** y seleccione el puerto donde está conectado el Arduino. En Windows típicamente aparece como **COM1**, **COM2** u otro número COM disponible.

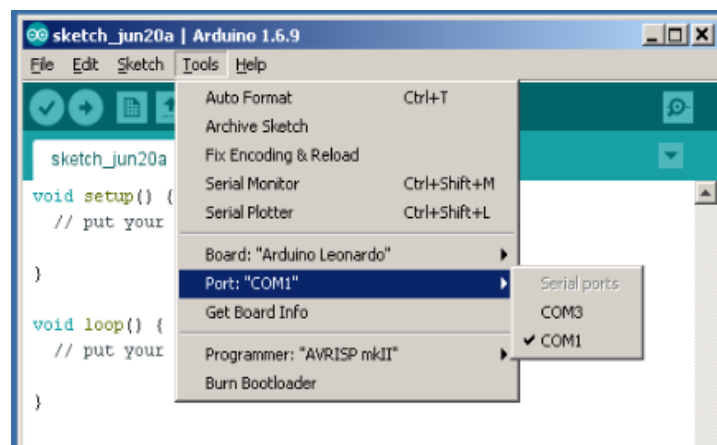


Figura 4. Selección del puerto serial.

3.4. Carga del Programa

Para cargar un programa en la tarjeta, presione el botón de **Cargar (Upload)** en la barra de herramientas:



- Mientras el programa se carga, los LED de comunicación Rx y Tx del Arduino parpadearán.
- Si la carga es exitosa, el IDE lo indicará en la barra de estado inferior.
- Pocos segundos después de finalizada la carga, el programa comenzará a ejecutarse automáticamente.

4. Estructura del IDE y de los Programas

El entorno de desarrollo del Arduino integra un editor de texto para escribir el código, un área de mensajes para los resultados de compilación y una consola de texto para la comunicación serial. Una vez que la tarjeta está conectada, el IDE se comunica con ella para cargar y depurar programas.

4.1. Sketches

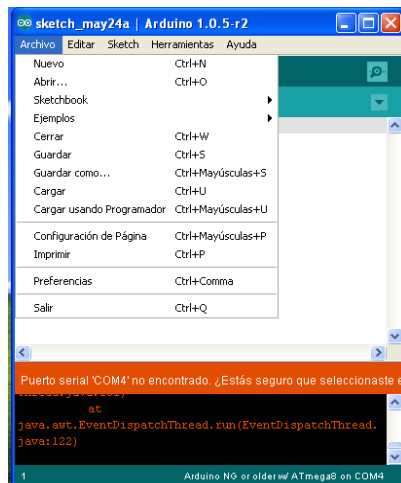
Los programas escritos para Arduino se denominan **sketches**. Se editan en el área de texto del IDE y se guardan con la extensión `.ino`. El área de mensajes (de fondo oscuro) muestra los resultados de la compilación y la comunicación con la tarjeta. Los botones de la barra superior permiten crear, abrir, compilar, cargar y guardar sketches.

4.2. Descripción de los Iconos de la Barra Principal

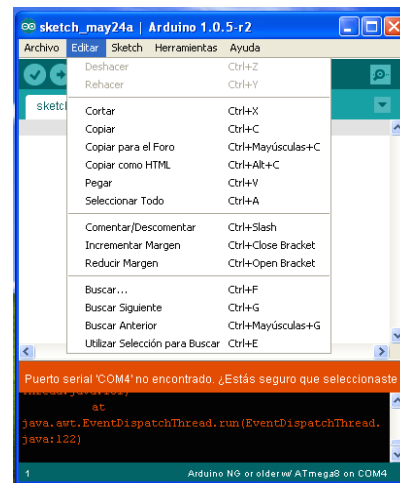
Icono	Nombre	Función
	Verify	Verifica que el código no contenga errores de sintaxis (compilación).
	Upload	Compila el código y lo carga al microcontrolador del Arduino.
	New	Crea un nuevo sketch en blanco.
	Open	Abre un sketch existente desde el menú de proyectos.
	Save	Guarda el sketch actual.
	Serial Monitor	Abre el Monitor Serial para visualizar la comunicación con el Arduino.

Adicionalmente, el IDE cuenta con cinco menús desplegables sensibles al contexto: Archivo, Editar, Sketch, Herramientas y Ayuda. Las siguientes figuras muestran los cuatro principales:

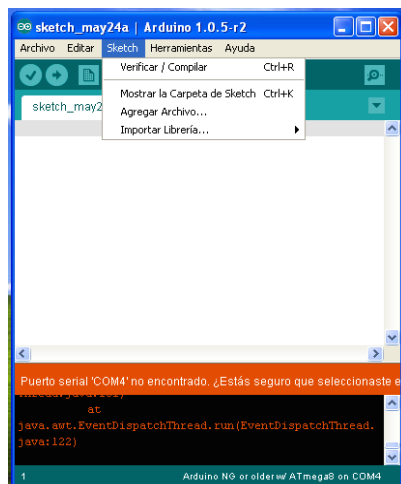
Menú Archivo



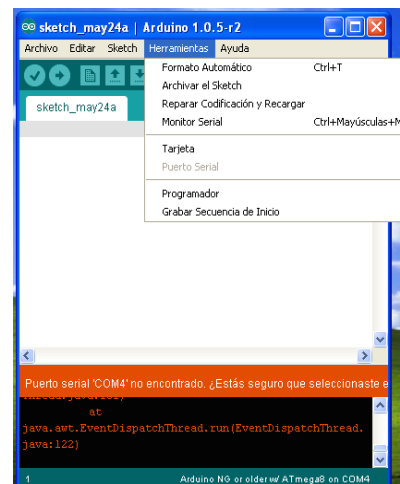
Menú Editar



Menú Sketch



Menú Herramientas



5. Estructura Básica de un Sketch

Todo sketch de Arduino se organiza alrededor de dos funciones obligatorias: `setup()` y `loop()`. Comprender su rol es esencial antes de escribir cualquier programa.

5.1. La función `setup()`

`setup()` es la primera función que ejecuta el Arduino. Se utiliza para inicializar variables, configurar el modo de los pines (entrada o salida), iniciar comunicaciones (serial, I²C, SPI) y cargar librerías. Esta función se ejecuta una sola vez, inmediatamente después del encendido o de un Reset de la tarjeta.

Analogía

Piense en **setup()** como el «encendido» de una máquina: solo se prepara una vez. Cuando usted enciende un automóvil, ajusta el espejo, abrocha el cinturón y enciende las luces antes de empezar a conducir; esa preparación inicial es el **setup()**.

5.2. La función loop()

Una vez ejecutado el **setup()**, el Arduino entra en la función **loop()**, que se ejecuta de forma indefinida en bucle. Aquí se coloca la lógica que el sistema debe repetir continuamente: leer sensores, encender actuadores, comunicarse con un PC, etc.

Analogía

Si **setup()** es el encendido del auto, **loop()** es la conducción: una y otra vez se observa el camino, se acelera, se frena y se gira el volante mientras el vehículo esté encendido.

5.3. Plantilla Básica

La siguiente plantilla no realiza ninguna acción, pero su estructura es la mínima necesaria para cualquier sketch. Conviene copiarla al inicio de cada nuevo programa.

```
void setup() {  
    // Coloque aquí el código que se ejecutará una sola vez:  
  
}  
  
void loop() {  
    // Coloque aquí el código que se repetirá en cada ciclo:  
  
}
```

Sintaxis: comentarios

Las líneas que comienzan con dos diagonales (//) son **comentarios**: el compilador las ignora. Sirven para documentar el código y para deshabilitar líneas temporalmente. Para comentarios de varias líneas se utilizan los delimitadores `/* ... */`.

6. Ejercicio 1 — Salida Digital (Do)

En este ejercicio se hará parpadear el LED interno del Arduino, conectado al pin 13.

6.1. Esquemático

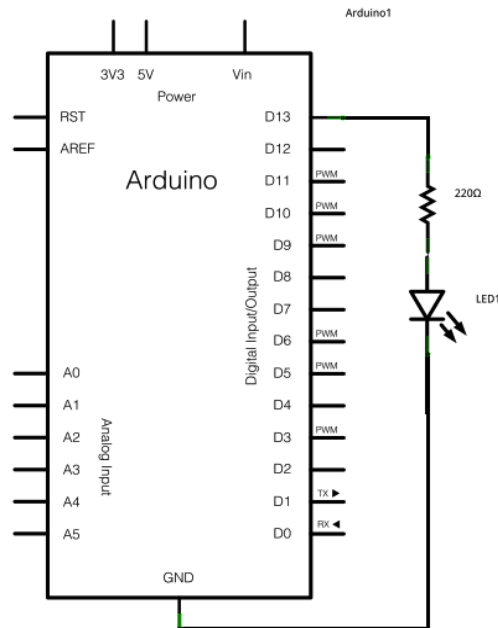


Figura 9. Esquemático del LED en el pin 13.

6.2. Procedimiento

1. En el IDE, vaya a Archivo → Ejemplos → 01.Basics → Blink.
2. Observe y analice cuidadosamente el código antes de cargarlo.
3. Cargue el programa al Arduino mediante el botón Upload.
4. El LED del pin 13 comenzará a parpadear con una frecuencia de un segundo encendido y un segundo apagado.
5. Modifique los tiempos a medio segundo (500 ms) y observe si hay cambio en el comportamiento del LED.
6. Desarrolle un programa de parpadeo de LED para otros puertos, encendiendo distintos LEDs (por ejemplo, los pines 11, 12 y 13). Utilice el siguiente esquema de conexión:

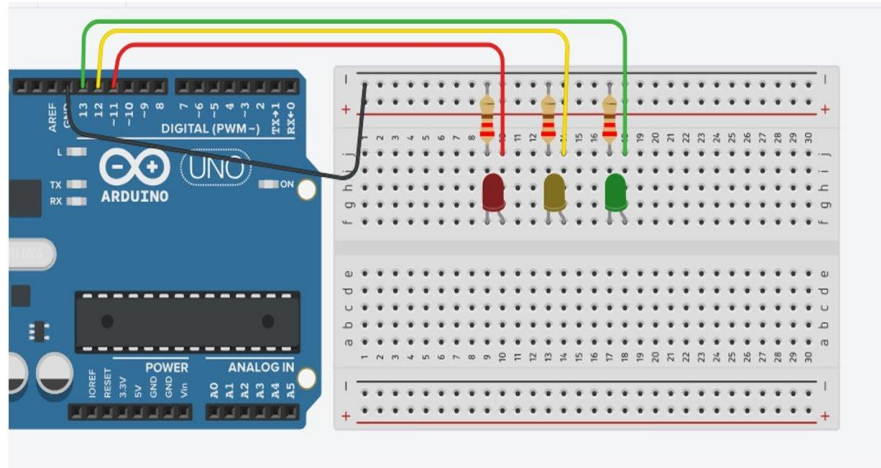


Figura 10. Conexión de tres LEDs a los pines 11, 12 y 13.

7. Modifique el programa para encender un LED cada segundo, otro cada 1.3 segundos y otro cada 2.1 segundos, utilizando únicamente los comandos `digitalWrite()` y `delay()`. Observe el comportamiento.

6.3. Comandos Utilizados

```
pinMode(13, OUTPUT);
```

Configura el pin 13 del Arduino como una salida digital. Es decir, el microcontrolador podrá imponer en ese pin un nivel alto (5 V) o bajo (0 V), pero no leer su estado.

 Nota

En el IDE actual, el pin 13 se identifica también con el nombre simbólico `LED_BUILTIN`, lo que hace al código más portable entre diferentes modelos de placas.

```
digitalWrite(13, HIGH);
```

Establece el pin 13 en nivel lógico alto (5 V). En el ejemplo Blink, esto enciende el LED.

```
delay(1000);
```

Pausa la ejecución del programa durante el tiempo indicado en milisegundos. En este caso, 1000 ms = 1 segundo.

```
digitalWrite(13, LOW);
```

Establece el pin 13 en nivel lógico bajo (0 V). En el ejemplo Blink, esto apaga el LED.

7. Ejercicio 2 — Entrada Digital (Di)

En este ejercicio se introduce la lectura del estado de un pulsador conectado a un pin de entrada digital.

7.1. Esquemático

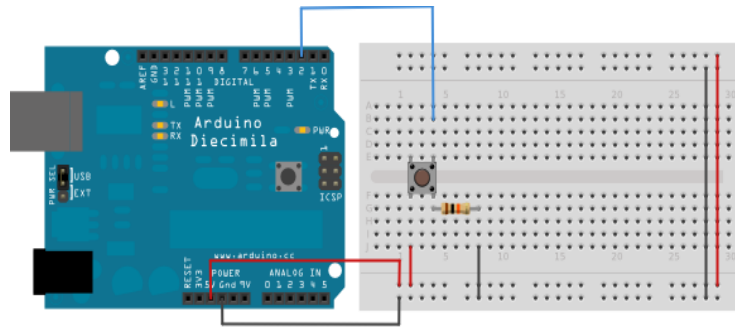


Figura 11. Conexión del pulsador al pin 2 con resistencia pull-down.

7.2. Procedimiento

1. Instale en el protoboard el circuito de la figura, conectando el pulsador al pin digital 2 del Arduino.
2. En el IDE, vaya a Archivo → Ejemplos → 02.Digital → Button.
3. Observe y analice el código antes de cargarlo.
4. Modifique el programa de tal forma que el LED del pin 13 parpadee cada 100 ms y que, al mantener pulsado el botón, parpadee cada un segundo.
5. Cargue el código en el Arduino y observe el comportamiento.

7.3. Comandos Utilizados

```
const int buttonPin = 2;
```

Define una constante de tipo entero llamada buttonPin con valor 2. Asignar nombres simbólicos a los pines mejora la legibilidad y el mantenimiento del código.

```
const int ledPin = 13;
```

Define la constante ledPin para el pin 13, donde se encuentra conectado el LED.

```
int buttonState = 0;
```

Declara una variable entera para almacenar el estado leído del pulsador (0 = no presionado, 1 = presionado).

```
pinMode(ledPin, OUTPUT);
```

Configura el pin del LED como salida.

```
pinMode(buttonPin, INPUT);
```

Configura el pin del pulsador como entrada digital.

💡 Concepto: const vs int

Una variable declarada con **const** **no puede modificarse** durante la ejecución del programa; sirve para valores que se determinan en tiempo de compilación, como un número de pin. Esto previene errores y permite al compilador optimizar el código.

8. Ejercicio 3 — Entrada Analógica (Ai)

En este ejercicio se utilizará el conversor analógico-digital (ADC) interno del Arduino para leer el voltaje variable provisto por un potenciómetro.

8.1. Esquemático

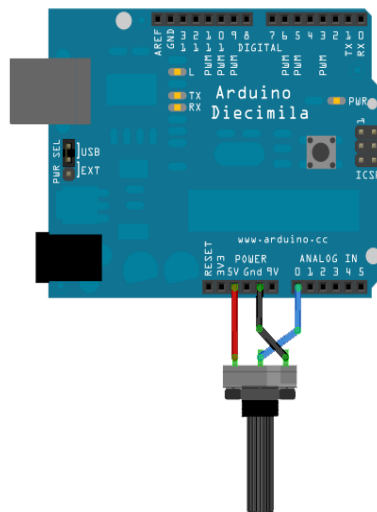


Figura 12. Conexión del potenciómetro al pin analógico A0.

8.2. Procedimiento

1. Instale en el Arduino el circuito de la figura, conectando los terminales fijos del potenciómetro a 5 V y a GND, y el terminal central al pin analógico A0.
2. En el IDE, vaya a Archivo → Ejemplos → 03.Analog → AnalogInput.
3. Observe y analice el código antes de cargarlo.
4. Cargue el código en el Arduino y observe qué ocurre cuando se gira la perilla del potenciómetro.

Concepto: resolución del ADC

El Arduino UNO posee un ADC de **10 bits**, lo que significa que entrega valores enteros entre 0 y 1023 (es decir, $2^{10} = 1024$ niveles distintos). Un valor de 0 corresponde a 0 V y uno de 1023 a 5 V (en el caso de `AREF = 5 V`).

9. Ejercicio 4 — Modulación por Ancho de Pulso (PWM)

La técnica PWM (Pulse Width Modulation) permite simular una salida analógica usando una salida digital. El Arduino UNO dispone de seis pines con capacidad PWM, identificados con el símbolo ~ junto a su número (pines 3, 5, 6, 9, 10 y 11).

Concepto: ¿qué es PWM?

Imagine que usted enciende y apaga rápidamente una bombilla muchas veces por segundo. Si la mantiene encendida la mitad del tiempo, percibirá un brillo equivalente al 50%. Eso es PWM: una señal digital que alterna entre 0 V y 5 V tan rápido que, ante cargas como un LED o un motor DC, se comporta como un voltaje promedio variable. La fracción del tiempo que la señal permanece en alto se denomina **ciclo de trabajo (duty cycle)**, expresado entre 0 y 255 en la función `analogWrite()` del Arduino.

9.1. Esquemático

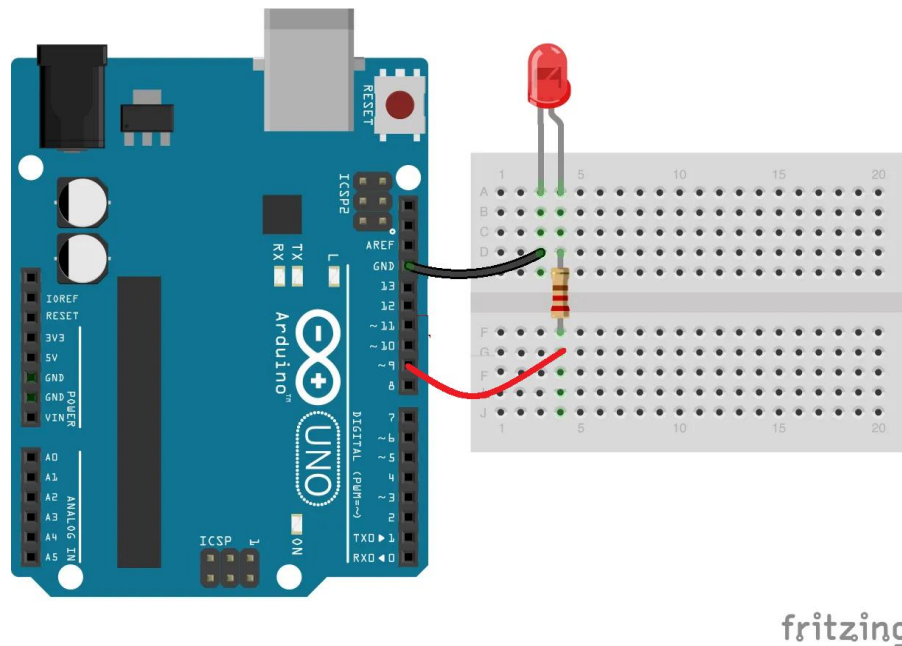


Figura 13. Conexión del LED al pin PWM 9.

9.2. Procedimiento

1. Conecte un LED al pin 9 del Arduino, en serie con una resistencia limitadora ($220\ \Omega - 330\ \Omega$) hacia GND.
2. En el IDE, vaya a Archivo → Ejemplos → 01.Basics → Fade.
3. Observe y analice el código antes de cargarlo.
4. Cargue el código y observe el comportamiento del LED.

10. Ejercicio 5 — Comunicación Serial

La comunicación serial es la forma más sencilla en que el Arduino intercambia datos con una computadora. Esto resulta esencial para depurar programas, visualizar valores de sensores y comandar la tarjeta desde el PC.

10.1. Esquemático

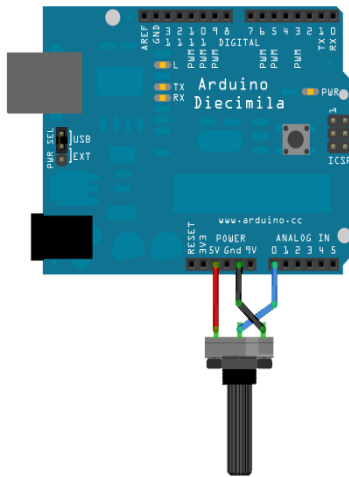


Figura 14. Conexión del potenciómetro al pin A0 (idéntica al Ejercicio 3).

10.2. Procedimiento

1. Mantenga el circuito del Ejercicio 3 (potenciómetro en el pin A0).
2. En el IDE, vaya a Archivo → Ejemplos → 01.Basics → AnalogReadSerial.
3. Observe y analice el código antes de cargarlo.
4. Cargue el código en el Arduino.
5. Abra el terminal de comunicación serial en Herramientas → Monitor Serial.
6. Observe el resultado en el terminal mientras varía el potenciómetro.
7. Modifique el tiempo de muestreo a medio segundo (500 ms) y observe el efecto.
8. Cierre el Monitor Serial y abra en su lugar el Serial Plotter desde Herramientas → Serial Plotter. Observe la diferencia respecto al Monitor Serial.

💡 Concepto: velocidad de transmisión (baud rate)

Tanto el Arduino como el terminal del PC deben acordar la velocidad de transmisión, expresada en **baudios** (bits por segundo). En el sketch se establece con `Serial.begin(9600);` y en el Monitor Serial debe seleccionarse el mismo valor en el menú desplegable inferior derecho. Si los valores no coinciden, los caracteres aparecerán como símbolos sin sentido.